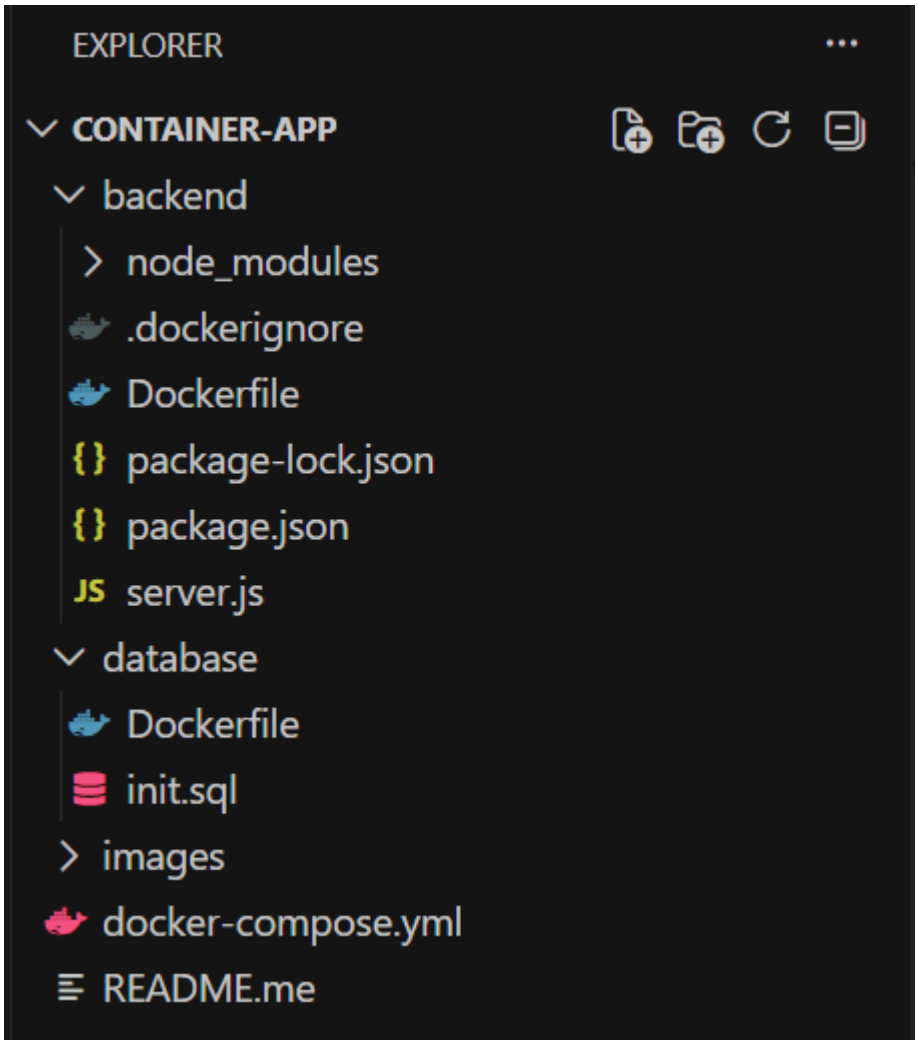
containerization-assignment-1

Assignment 1 – Containerized Web Application with PostgreSQL using Docker Compose and Macvlan

This assignment demonstrates how to build and deploy a containerized Node.js web application connected to a PostgreSQL database using **Docker Compose**.
The setup uses a **Macvlan network** to assign static IP addresses to containers and ensures **data persistence using Docker volumes**.

Project Structure

```
assignment-1-containerized-webapp
│
├── backend
│   ├── server.js
│   ├── Dockerfile
│   ├── package.json
│   └── .dockerignore
│
├── database
│   ├── Dockerfile
│   └── init.sql
│
├── docker-compose.yml
└── README.md
```



Pre-requisite

- Docker installed
- Docker Compose installed
- Node.js installed
- Linux / WSL environment

Step 1 – Initialize Node Package

```
npm init -y
```

```
sristee@Sristee:/mnt/c/Users/srist/container-app$ cd backend
sristee@Sristee:/mnt/c/Users/srist/container-app/backend$ npm init -y
Wrote to /mnt/c/Users/srist/container-app/backend/package.json:

{
  "name": "backend",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

Step 2 – Install Required Packages

```
npm i express pg
```

```
sristee@Sristee:/mnt/c/Users/srist/container-app/backend$ npm i express pg

added 79 packages, and audited 80 packages in 2s

22 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
sristee@Sristee:/mnt/c/Users/srist/container-app/backend$
```

Step 3 – package.json

After installing dependencies the `package.json` file will look like this.

```
Wrote to /mnt/c/Users/srist/container-app/backend/package.json:

{
  "name": "backend",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}
```

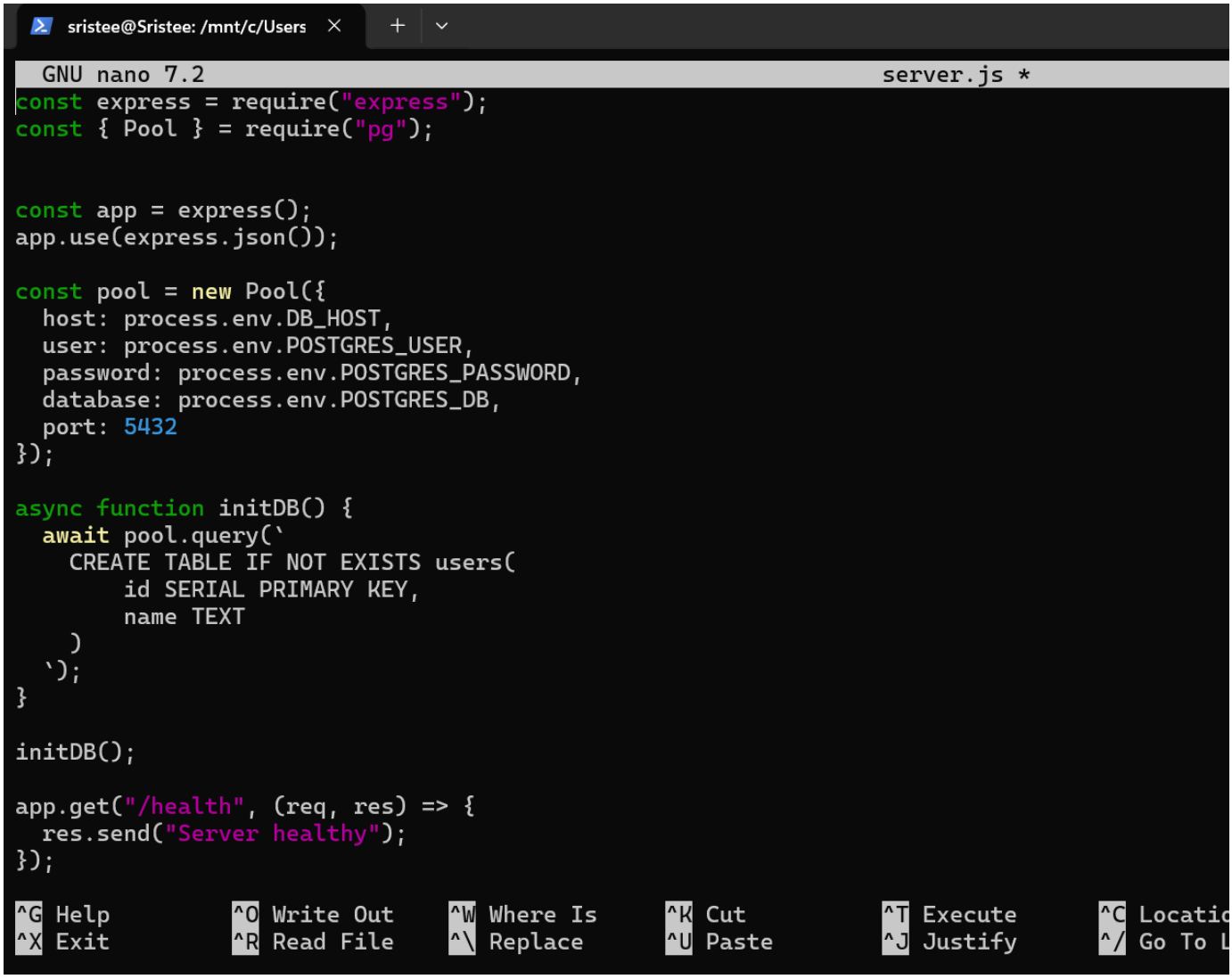
Step 4 – Create Backend Server

Create `server.js`

```
sristee@Sristee:/mnt/c/Users/srist/container-app/backend$ nano server.js
```

```
const express = require("express");
const { Pool } = require("pg");

const app = express();
app.use(express.json());

const pool = new Pool({
  host: process.env.DB_HOST,
  user: process.env.POSTGRES_USER,
  password: process.env.POSTGRES_PASSWORD,
  database: process.env.POSTGRES_DB,
  port: 5432
});

async function initDB() {
  await pool.query(`
    CREATE TABLE IF NOT EXISTS users(
        id SERIAL PRIMARY KEY,
        name TEXT
    )
  `);
}

initDB();
```

```
app.get("/health", (req, res) => {
  res.send("Server healthy");
});

app.post("/users", async (req, res) => {
  const { name } = req.body;

  const result = await pool.query(
    "INSERT INTO users(name) VALUES($1) RETURNING *",
    [name]
  );

  res.json(result.rows[0]);
});

app.get("/users", async (req, res) => {
  const result = await pool.query("SELECT * FROM users");
  res.json(result.rows);
});

app.listen(3000, "0.0.0.0", () => {
  console.log("Server running on port 3000");
});
```



Step 5 – Backend Dockerfile

Create `backend/Dockerfile`

```
sristee@Sristee:/mnt/c/Users/srist/container-app/backend$ nano Dockerfile
```

```
# Builder Stage
FROM node:20-alpine AS builder

WORKDIR /app

COPY package*.json ./

RUN npm install --only=production

COPY . .

# Runtime Stage
FROM node:20-alpine

WORKDIR /app

RUN addgroup -S appgroup && adduser -S appuser -G appgroup

COPY --from=builder /app .

USER appuser

EXPOSE 3000

CMD ["node", "src/server.js"]
```

```
sristee@Sristee: /mnt/c/Users
  GNU nano 7.2                                                  Dockerfile *
# Builder Stage
FROM node:20-alpine AS builder

WORKDIR /app

COPY package*.json ./

RUN npm install --only=production

COPY . .

# Runtime Stage
FROM node:20-alpine

WORKDIR /app

RUN addgroup -S appgroup && adduser -S appuser -G appgroup

COPY --from=builder /app .

USER appuser

EXPOSE 3000

CMD ["node", "src/server.js"]

^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C Locati
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To
```

Step 6 – .dockerignore

Create `.dockerignore`

```
sristee@Sristee:/mnt/c/Users/srist/container-app/backend$ nano .dockerignore
```

```
node_modules
npm-debug.log
Dockerfile
.git
.gitignore
```

```
sristee@Sristee: /mnt/c/Users
  GNU nano 7.2
node_modules
npm-debug.log
Dockerfile
.git
.gitignore
```

Step 7 – Database Dockerfile

Create `database/Dockerfile`

```
sristee@Sristee:/mnt/c/Users/srist/container-app/database$ nano Dockerfile
```

```
FROM postgres:15-alpine

COPY init.sql /docker-entrypoint-initdb.d/
```

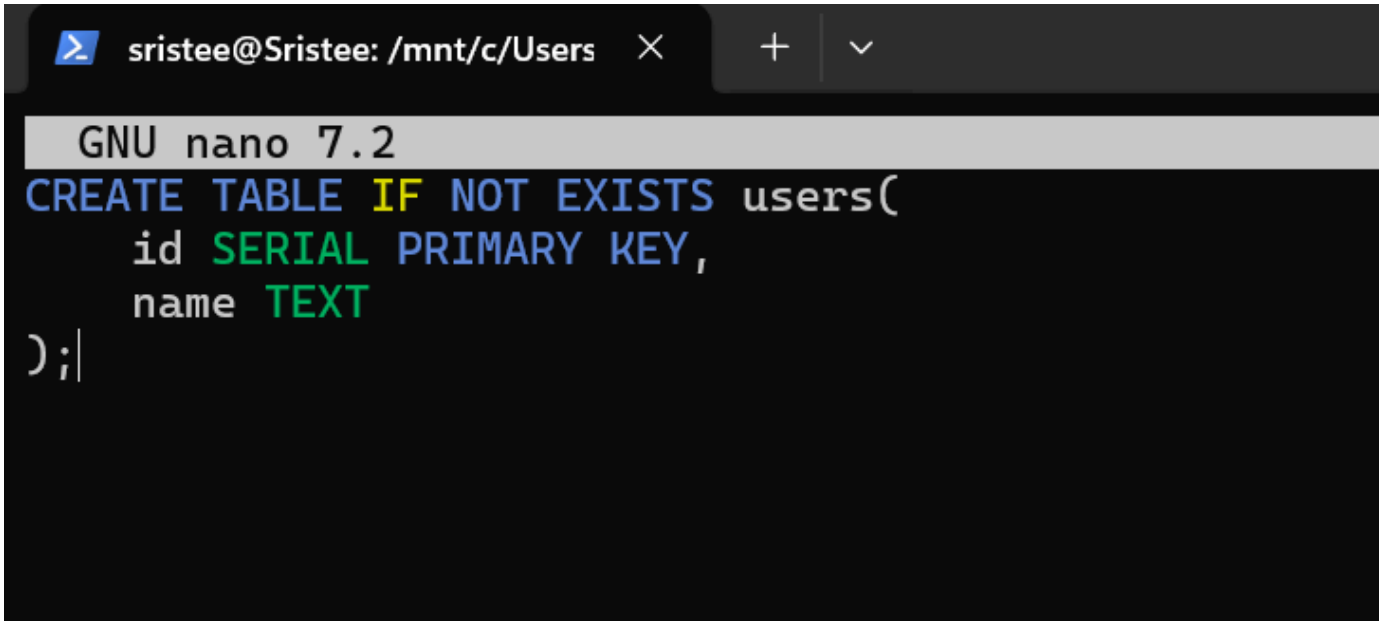
```
sristee@Sristee: /mnt/c/Users
  GNU nano 7.2
FROM postgres:15-alpine
COPY init.sql /docker-entrypoint-initdb.d/
```

Step 8 – Database Initialization Script

Create `init.sql`

```
sristee@Sristee:/mnt/c/Users/srist/container-app/database$ nano init.sql
```

```sql
CREATE TABLE IF NOT EXISTS users(
    id SERIAL PRIMARY KEY,
    name TEXT
);
```



Step 9 – Docker Compose Configuration

Create `docker-compose.yml`

```
sristee@Sristee:/mnt/c/Users/srist/container-app/database$ cd ..
sristee@Sristee:/mnt/c/Users/srist/container-app$ nano docker-compose.yml
```

```yaml
version: "3.9"

services:

  database:
    build: ./database
    container_name: postgres_db
    restart: always

    environment:
      POSTGRES_DB: mydb
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: dhairya
```

```yaml
    volumes:
      - pgdata:/var/lib/postgresql/data

    networks:
      macvlan_net:
        ipv4_address: 192.168.50.21

    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U admin"]
      interval: 10s
      retries: 5


  backend:
    build: ./backend
    container_name: node_backend
    restart: always

    environment:
      DB_HOST: 192.168.50.21
      POSTGRES_DB: mydb
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: dhairya

    depends_on:
      database:
        condition: service_healthy

    networks:
      macvlan_net:
        ipv4_address: 192.168.50.20


volumes:
  pgdata:

networks:
  macvlan_net:
    external: true
```

```
sristee@Sristee: /mnt/c/Users

  GNU nano 7.2                                          docker-compose.yml *
version: "3.9"

services:

  database:
    build: ./database
    container_name: postgres_db
    restart: always

    environment:
      POSTGRES_DB: mydb
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: dhairya

    volumes:
      - pgdata:/var/lib/postgresql/data

    networks:
      macvlan_net:
        ipv4_address: 192.168.50.21

    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U admin"]
      interval: 10s
      retries: 5


  backend:
    build: ./backend

^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C Location
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To Lin
```

Step 10 – Find Network Interface

```
ip a
```

```
sristee@Sristee:/mnt/c/Users/srist/container-app$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet 10.255.255.254/32 brd 10.255.255.254 scope global lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 00:15:5d:9b:4e:a5 brd ff:ff:ff:ff:ff:ff
    inet 172.29.194.31/20 brd 172.29.207.255 scope global eth0
       valid_lft forever preferred_lft forever
    inet6 fe80::215:5dff:fe9b:4ea5/64 scope link
       valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether c6:93:9c:59:53:0c brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever
4: br-370a2053131a: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 86:46:e9:89:f6:33 brd ff:ff:ff:ff:ff:ff
    inet 172.18.0.1/16 brd 172.18.255.255 scope global br-370a2053131a
       valid_lft forever preferred_lft forever
5: docker_gwbridge: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether e6:df:f3:c5:af:b6 brd ff:ff:ff:ff:ff:ff
    inet 172.19.0.1/16 brd 172.19.255.255 scope global docker_gwbridge
       valid_lft forever preferred_lft forever
    inet6 fe80::e4df:f3ff:fec5:afb6/64 scope link
       valid_lft forever preferred_lft forever
15: veth818f10c@if14: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master docker_gwbridge state UP group default
    link/ether 4e:73:40:2c:b2:69 brd ff:ff:ff:ff:ff:ff link-netnsid 1
    inet6 fe80::4c73:40ff:fe2c:b269/64 scope link
       valid_lft forever preferred_lft forever
```

Step 11 – Create Macvlan Network

```
docker network create -d macvlan \
--subnet=192.168.50.0/24 \
--gateway=192.168.50.1 \
-o parent=eth0 \
macvlan_net
```

```
sristee@Sristee:/mnt/c/Users/srist/container-app$ docker network create -d macvlan \
--subnet=192.168.50.0/24 \
--gateway=192.168.50.1 \
-o parent=eth0 \
macvlan_net
c594085811cca60a94eb04bf8d7bb7875e48df5cf4f058b4c5cd4105c899967e
```

Step 12 – Build Containers

```
docker-compose up --build --no-cache
```

```
^Csristee@Sristee:/mnt/c/Users/srist/container-app$ docker-compose up --build --no-cache
Builds, (re)creates, starts, and attaches to containers for a service.

Unless they are already running, this command also starts any linked services.

The `docker-compose up` command aggregates the output of each container. When
the command exits, all containers are stopped. Running `docker-compose up -d`
starts the containers in the background and leaves them running.

If there are existing containers for a service, and the service's configuration
or image was changed after the container's creation, `docker-compose up` picks
up the changes by stopping and recreating the containers (preserving mounted
volumes). To prevent Compose from picking up changes, use the `--no-recreate`
flag.

If you want to force Compose to stop and recreate all containers, use the
`--force-recreate` flag.

Usage: up [options] [--scale SERVICE=NUM...] [--] [SERVICE...]

Options:
    -d, --detach               Detached mode: Run containers in the background,
                               print new container names. Incompatible with
                               --abort-on-container-exit.
    --no-color                 Produce monochrome output.
    --quiet-pull               Pull without printing progress information
    --no-deps                  Don't start linked services.
    --force-recreate           Recreate containers even if their configuration
                               and image haven't changed.
    --always-recreate-deps     Recreate dependent containers.
                               Incompatible with --no-recreate.
    --no-recreate              If containers already exist, don't recreate
                               them. Incompatible with --force-recreate and -V.
```

Step 13 – Start Services

```
docker-compose up -d
```

```
sristee@Sristee:/mnt/c/Users/srist/container-app$ docker-compose up -d
WARNING: The Docker Engine you're using is running in swarm mode.

Compose does not use swarm mode to deploy services to multiple nodes in a swarm. All containers will be scheduled on the current node
.

To deploy your application across the swarm, use `docker stack deploy`.

Building database
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  3.072kB
Step 1/2 : FROM postgres:15-alpine
15-alpine: Pulling from library/postgres
589002ba0eae: Already exists
aa104f8e5799: Pulling fs layer
ac1ff2379be8: Pulling fs layer
8c9e2aa22b4c: Pulling fs layer
765f80b9a483: Pulling fs layer
a27cb555d458: Pulling fs layer
610258b635a8: Pulling fs layer
2a9c4fd442cb: Pulling fs layer
9f6d6f38d9d3: Pulling fs layer
b53228f7876a: Pulling fs layer
34c88efd36aa: Pulling fs layer
610258b635a8: Waiting
765f80b9a483: Waiting
2a9c4fd442cb: Waiting
```

```
Creating postgres_db ... done
Creating node_backend ... done
sristee@Sristee:/mnt/c/Users/sri
```

Step 14 – Insert User Using API

```
curl -X POST http://192.168.50.20:3000/users \
-H "Content-Type: application/json" \
-d '{"name":"Sristee"}'
```

```
sristee@Sristee:/mnt/c/Users/srist/container-app$ curl -X POST http://192.168.50.20:3000/users \
-H "Content-Type: application/json" \
-d '{"name":"Sristee"}'
```

Step 15 – Get Users

```
curl http://192.168.50.20:3000/users
```

```
sristee@Sristee:/mnt/c/Users/srist/container-app$ curl http://192.168.50.20:3000/users
```

Step 16 – List Running Containers

```
docker ps
```

```
sristee@Sristee:/mnt/c/Users/srist/container-app$ docker ps
CONTAINER ID   IMAGE                    COMMAND                  CREATED         STATUS                          PORTS     NAMES
c01e378b97c7   container-app_backend    "docker-entrypoint.s…"   3 minutes ago   Restarting (1) 21 seconds ago             node_backe
nd
493de4b76598   container-app_database   "docker-entrypoint.s…"   3 minutes ago   Up 3 minutes (healthy)                    postgres_d
b
```

Step 17 – List Docker Volumes

```
docker volume ls
```

```
sristee@Sristee:/mnt/c/Users/srist/container-app$ docker volume ls
DRIVER    VOLUME NAME
local     35e0ad5bdfc326fc63153ca154c54cfb70b3b012674c3555676ca6f6b5f8299d
local     6573f937d22e216819ed0b2d87729eee10be67aa2243c63505eee740d1d37709
local     container-app_pgdata
local     d58d4af4c7ec391077aa6bf7a1e0f8ef27cf8a0dad4fffc8277dd681cf8314b4
local     myvolume
```

Step 18 – Inspect Network

```
docker network inspect macvlan_net
```

```
sristee@Sristee:/mnt/c/Users/srist/container-app$ docker network inspect macvlan_net
[
    {
        "Name": "macvlan_net",
        "Id": "c594085811cca60a94eb04bf8d7bb7875e48df5cf4f058b4c5cd4105c899967e",
        "Created": "2026-03-16T16:24:50.37306578Z",
        "Scope": "local",
        "Driver": "macvlan",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "192.168.50.0/24",
                    "Gateway": "192.168.50.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Containers": {
            "493de4b765985f46c409b7a62a9350103d2d3f0bd8b942641ef651f958d21582": {
                "Name": "postgres_db",
                "EndpointID": "1c7cb9a2b15bcc7d17c9bbf8aae8b32cc287023eeb54cab0f2ba0385246dd685",
                "MacAddress": "fa:9a:07:ec:d1:73",
                "IPv4Address": "192.168.50.21/24",
```

Step 19 – Inspect Backend Container

```
docker inspect node_backend
```

```
sristee@Sristee:/mnt/c/Users/srist/container-app$ docker inspect node_backend
[
    {
        "Id": "c01e378b97c7335a684c6e41f62fcd1785e1adc2ea4e2bff0f9b1581cd1348b3",
        "Created": "2026-03-16T16:41:33.550799002Z",
        "Path": "docker-entrypoint.sh",
        "Args": [
            "node",
            "src/server.js"
        ],
        "State": {
            "Status": "restarting",
            "Running": true,
            "Paused": false,
            "Restarting": true,
            "OOMKilled": false,
            "Dead": false,
            "Pid": 0,
            "ExitCode": 1,
            "Error": "",
            "StartedAt": "2026-03-16T16:44:11.866678984Z",
            "FinishedAt": "2026-03-16T16:44:12.450493339Z"
        },
        "Image": "sha256:4f1284b14e3851e84f3f6da5a142f15f869b5031727d4a2bf31acfc6fbfab41b",
        "ResolvConfPath": "/var/lib/docker/containers/c01e378b97c7335a684c6e41f62fcd1785e1adc2ea4e2
        "HostnamePath": "/var/lib/docker/containers/c01e378b97c7335a684c6e41f62fcd1785e1adc2ea4e2b
        "HostsPath": "/var/lib/docker/containers/c01e378b97c7335a684c6e41f62fcd1785e1adc2ea4e2bff0
        "LogPath": "/var/lib/docker/containers/c01e378b97c7335a684c6e41f62fcd1785e1adc2ea4e2bff0f9
e41f62fcd1785e1adc2ea4e2bff0f9b1581cd1348b3-json.log",
        "Name": "/node_backend",
        "RestartCount": 12,
        "Driver": "overlay2",
        "Platform": "linux",
```

Step 20 – Inspect Database Container

```
docker inspect postgres_db
```

```
sristee@Sristee:/mnt/c/Users/srist/container-app$ docker inspect postgres_db
[
    {
        "Id": "493de4b765985f46c409b7a62a9350103d2d3f0bd8b942641ef651f958d21582",
        "Created": "2026-03-16T16:41:22.945322515Z",
        "Path": "docker-entrypoint.sh",
        "Args": [
            "postgres"
        ],
        "State": {
            "Status": "running",
            "Running": true,
            "Paused": false,
            "Restarting": false,
            "OOMKilled": false,
            "Dead": false,
            "Pid": 2710,
            "ExitCode": 0,
            "Error": "",
            "StartedAt": "2026-03-16T16:41:23.057073519Z",
            "FinishedAt": "0001-01-01T00:00:00Z",
            "Health": {
                "Status": "healthy",
                "FailingStreak": 0,
                "Log": [
                    {
                        "Start": "2026-03-16T16:44:24.508670471Z",
                        "End": "2026-03-16T16:44:24.626105784Z",
                        "ExitCode": 0,
                        "Output": "/var/run/postgresql:5432 - accepting connections\n"
                    },
                    {
                        "Start": "2026-03-16T16:44:34.626730199Z",
```

Step 21 – Verify Data Persistence

```
docker-compose down
docker-compose up -d
```

```
sristee@Sristee:/mnt/c/Users/srist/container-app$ docker-compose down
Stopping node_backend ... done
Stopping postgres_db  ... done
Removing node_backend ... done
Removing postgres_db  ... done
Network macvlan_net is external, skipping
sristee@Sristee:/mnt/c/Users/srist/container-app$ docker-compose up -d
WARNING: The Docker Engine you're using is running in swarm mode.

Compose does not use swarm mode to deploy services to multiple nodes in a swarm. All containers will be scheduled on the current node
.

To deploy your application across the swarm, use `docker stack deploy`.

Creating postgres_db ... done
Creating node_backend ... done
```

Build Optimization Explanation

Several optimization techniques were used while building the Docker images.

Multi-stage build was used to separate dependency installation from runtime execution. This reduces the final image size by excluding unnecessary build tools.

A lightweight **node:20-alpine** base image was chosen instead of the standard Node.js image, which significantly reduces image size and improves container startup speed.

A `.dockerignore` file prevents unnecessary files such as `node_modules` and `.git` from being copied into the container image.

Finally, the container runs using a **non-root user**, improving application security.

Image Size Comparison

Image	Size
node:20	~1.1 GB
node:20-alpine	~180 MB

Using Alpine significantly reduces storage requirements and download time.

Macvlan vs IPvlan

Feature	MACVLAN	IPVLAN
MAC addresses	One per container	One shared
Network switch load	Higher	Lower
Scalability	Limited by switch	Higher
Best for	Small deployments	Large-scale

Conclusion

This assignment demonstrates how to containerize a full-stack application using Docker and Docker Compose.
It also highlights advanced networking using Macvlan and ensures persistent database storage using Docker volumes.